

Inner Exception in C# with an Example

In this article, I am going to discuss the **Inner Exception in C#** with Examples. Please read our previous article where we discussed [How to Create a Custom Exception in C#](#) with examples. At the end of this article, you will understand what Inner Exception is and its need in C# with Examples.

What is Inner Exception in C#?

The Inner Exception in C# is a property of the Exception class. If you go to the definition of the Exception class, then you will see that it is a read-only property i.e. having only get accessor as shown in the below image. As this property is defined in the parent Exception class, so this property is available to all the Child classes including the Custom Exception classes.

```
...public class Exception : ISerializable, _Exception
{
    ...public Exception();
    ...public Exception(string message);
    ...public Exception(string message, Exception innerException);
    ...protected Exception(SerializationInfo info, StreamingContext context);

    ...public virtual string Source { get; set; }
    ...public virtual string HelpLink { get; set; }
    ...public virtual string StackTrace { get; }
    ...public MethodBase TargetSite { get; }
    ...public Exception InnerException { get; }
    ...public virtual string Message { get; }
    ...public int HResult { get; protected set; }
    ...public virtual IDictionary Data { get; }

    ...protected event EventHandler<SafeSerializationEventArgs> SerializeObjectState;

    ...public virtual Exception GetBaseException();
    ...public virtual void GetObjectData(SerializationInfo info, StreamingContext context);
    ...public Type GetType();
    ...public override string ToString();
}
```

public Exception InnerException {get;}: This InnerException property gets the Exception instance that caused the current exception. It returns an object that describes the error that caused the current exception. The InnerException property returns the same value that passed into the constructor, or null if the inner exception value was not supplied to the constructor.

To simplify the above definition, we can say that, when there is a series of exceptions, then the **Most Current Exception Obtains the Previous Exception Details in the InnerException Property**. Suppose, in our application, from Method1, we are calling Method2. In Method2, we are getting one exception let's say divide by zero exception, and then from Method1 we are getting another exception, let's say Format exception. Then, in this case, the current exception or the latest exception is Format Exception and in the Format Exception InnerException property, you will get the previous exception details i.e. Divide By Zero Exception.

In other words, we can say that the `InnerException` property returns the original exception that caused the current exception. If this is not clear at the moment, then don't worry we will discuss this with examples.

Inner Exception Example in C#:

Let us say we have an exception inside a try block that is throwing `DivideByZeroException` and the catch block catches that exception and then tries to write that exception to a file. However, if the file path is not found, then the catch block is also going to throw `FileNotFoundException`.

Let's say the outside try block catches this `FileNotFoundException` exception, but how about the actual `DivideByZeroException` that was thrown? Is it lost? No, the `InnerException` property of the Exception class contains the actual exception.

Example to Understand Inner Exception in C#:

Let us understand the Inner Exception in C# with an example. Let us first see the complete Example Code first and then I will explain the code. The complete code to understand Inner Exception in C# is given below:

```
using System;
using System.IO;
using System.Text;
namespace ExceptionHandlingDemo
{
    class Program
    {
        public static void Main()
        {
            //Outer Try
            try
            {
                int FirstNumber, SecondNumber, Result;
                //Inner Try
                try
                {
                    //Make sure to Cause Exception in the Try Block
                    Console.WriteLine("Enter First Number:");
                    FirstNumber = Convert.ToInt32(Console.ReadLine());
                    Console.WriteLine("Enter Second Number:");
                    SecondNumber = Convert.ToInt32(Console.ReadLine());
                    Result = FirstNumber / SecondNumber;
                    Console.WriteLine($"Result = {Result}");
                }
                //Inner Catch
                catch (Exception ex)
                {
                    //Make sure this Path Does Not Exist
                    string filePath = @"D:\Projects\LogFile\Log.txt";
                    if (File.Exists(filePath))
                    {
                        StringBuilder stringBuilder = new StringBuilder();
```

```

        stringBuilder.Append($"Message: {ex.Message} \n");
        stringBuilder.Append($"Source: {ex.Source} \n");
        stringBuilder.Append($"HelpLink: {ex.HelpLink} \n");
        stringBuilder.Append($"StackTrace: {ex.StackTrace} \n");
        stringBuilder.Append($"GetType(): {ex.GetType()} \n");
        stringBuilder.Append($"GetType().Name: {ex.GetType().Name} \n");

        StreamWriter streamWriter = new StreamWriter(filePath);
        streamWriter.Write(stringBuilder.ToString());
        streamWriter.Close();
        Console.WriteLine("There is a Problem! Please Try Later");
    }
    else
    {
        //To retain the Original Exception pass, this exception
parameter
        //to the constructor of the current exception
        string Message = filePath + " Does Not Exist";
        throw new FileNotFoundException(Message, ex);
    }
}
//Outer Catch
catch (Exception exception)
{
    //exception.Message will give the current exception message
    //i.e. Message about File Not Found Exception
    Console.WriteLine("\nCurrent Exception Details: ");
    Console.WriteLine($"Current Exception Message: {exception.Message}");
    Console.WriteLine($"Current Exception Source: {exception.Source}");
    Console.WriteLine($"Current Exception StackTrace:
{exception.StackTrace}");
    //Check if InnerException is not null before accessing the Inner
properties
    //else, you may get Null Reference Exception
    if (exception.InnerException != null)
    {
        Console.WriteLine("\nInner Exception Details: ");
        Console.WriteLine($"Inner Exception Message:
{exception.InnerException.Message}");
        Console.WriteLine($"Inner Exception Source:
{exception.InnerException.Source}");
        Console.WriteLine($"Inner Exception StackTrace:
{exception.InnerException.StackTrace}");
    }
    Console.WriteLine("Main Method End");
    Console.ReadLine();
}
}
}

```

Point 1: First we are asking the user to enter two numbers. In order to understand Inner Exception, we have to make sure this program causes an exception while running the application. To do that we have 3 options

1. You can enter a Character instead of a number which will cause a Format Exception.
2. Or, you can enter a very big number that an integer cannot hold which will cause Over Flow Exception.
3. Or, you can enter zero for the Second Number which will cause the application to throw the Divide By Zero Exception.

Point 2: Once you cause your application to throw an error from the inner try block, then that error is going to be handled by the Inner Catch block. This is because the inner catch is a generic catch block, taking the Exception class as a parameter that can capture any type of exceptions thrown from the corresponding try block.

Point 3: Once the catch block catches the exception, then we are trying to log the exception details into a text file. Here, if you are providing the correct file path, then the exception information will be logged into the text file. But to understand Inner Exception, make sure that the file path does not exist. If the File Path does not exist, then we are throwing a File Not Found Exception from the catch block and if you see, we are passing two parameters to the constructor of the File Not Found Exception class. The first parameter specifies the message and the second parameter is the exception (the exception that was thrown from the inner try block) and this exception information will be stored inside the InnerException property.

Point 4: Now, the outer catch block will catch the File Not Found Exception which is thrown by the inner catch block. Here, first, we are printing the current exception details and then we are printing the original or the old exception details i.e. the exception which is originally thrown from the Inner Try block. And we can access the old exception details from the Inner Exception property. But before accessing the InnerException properties, please make sure that the InnerException value is not null, else you might get a Null Reference Exception.

Output1:

Now, run the application and provide the two input numbers as 100 and 0, this time, the inner try block will throw Divide By Zero Exception and you will see that this Divide By Zero Exception details will be printed by Inner Exception as shown in the below image.

```
Enter First Number:
100
Enter Second Number:
0

Current Exception Details:
Current Exception Message: D:\Projects\LogFile\Log.txt Does Not Exist
Current Exception Source: ExceptionHandlingDemo
Current Exception StackTrace:    at ExceptionHandlingDemo.Program.Main() in D:\Projects\ExceptionHandlingDemo\
ExceptionHandlingDemo\Program.cs:line 53

Inner Exception Details:
Inner Exception Message: Attempted to divide by zero.
Inner Exception Source: ExceptionHandlingDemo
Inner Exception StackTrace:    at ExceptionHandlingDemo.Program.Main() in D:\Projects\ExceptionHandlingDemo\Ex
ceptionHandlingDemo\Program.cs:line 25
Main Method End
```

Output2:

Now, run the application and provide the second value as abc and this time, the inner try block will throw Format Exception and you will see that this Format Exception details will be printed by Inner Exception as shown in the below image.

```
Enter First Number:
100
Enter Second Number:
abc

Current Exception Details:
Current Exception Message: D:\Projects\LogFile\Log.txt Does Not Exist
Current Exception Source: ExceptionHandlingDemo
Current Exception StackTrace:    at ExceptionHandlingDemo.Program.Main() in D:\Projects\ExceptionHandlingDemo\B
ExceptionHandlingDemo\Program.cs:line 53

Inner Exception Details:
Inner Exception Message: Input string was not in a correct format.
Inner Exception Source: mscorlib
Inner Exception StackTrace:    at System.Number.StringToNumber(String str, NumberStyles options, NumberBuffer&
number, NumberFormatInfo info, Boolean parseDecimal)
    at System.Number.ParseInt32(String s, NumberStyles style, NumberFormatInfo info)
    at System.Convert.ToInt32(String value)
    at ExceptionHandlingDemo.Program.Main() in D:\Projects\ExceptionHandlingDemo\ExceptionHandlingDemo\Program.c
s:line 23
Main Method End
```

In the next article, I am going to discuss [Exception Handling Abuse](#) in C#. Here, in this article, I try to explain the **Inner Exception in C#** with examples. I would like to have your feedback. Please post your feedback, question, or comments about this article.